```
################################################################################################################################
Red Hat Certified Engineer (RHCE) EX294 -
################################################################################################################################
################################################################################################################################
LESSON #1 - INSTALLATION OF ANSIBLE
################################################################################################################################
++++++++++++++++++++++++++++++++1.1 Understanding Ansible++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++1.2 Host Requirements++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++1.3 Installing Ansible on the Control Node++++++++++++++++++++++++++++++++
CENTOS-8 Install
STEP1 - ENABLE EPEL REPO
    yum install epel-release
STEP2 - INSTALL Ansible
    yum install ansible -y
STEP3 - ADD A NEW USER FOR ANSIBLE
    useradd devops; passwd devops (All the nodes)
STEP4 - ADD THE ANSIBLE USER IN SUDO WITH NOPASSWD Option in all nodes
    useradd devops; passwd devops
    visudo
    devops ALL=(ALL)    NOPASSWD: ALL
    visudo -c

STEP5 - ENABLE PASSWORDLESS AUTH FOR ANSIBLE USER ACROSS ALL MANAGED NODES
    login to user devops and run the commands below
    ssh-keygen
    ssh-copy-id devops@node1
    ssh-copy-id devops@node2
    ssh-copy-id devops@node3




yum install https://dl.fedoraproject.org/pub/epel/epel-release-latest-8.noarch.rpm

yum install ansible

yum list installed python(RHEL7)

yum list installed platform-python(RHEL8)

yum install ansible

ansible --version

ansible 2.7.1
  config file = /etc/ansible/ansible.cfg
  configured module search path = [u'/home/student/.ansible/plugins/modules', u'/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python2.7/site-packages/ansible
  executable location = /usr/bin/ansible
  python version = 2.7.5 (default, Sep 12 2018, 05:31:16) [GCC 4.8.5 20150623 (Red Hat 4.8.5-36)]

ansible -m setup localhost | grep ansible_python_version

dedticated non root user account + ssh keys + sudo configured in order to connect to managed hosts

manage nodes entries in /etc/hosts

++++++++++++++++++++++++++++++++1.4 Preparing Managed Nodes++++++++++++++++++++++++++++++++
```

```
yum list installed python3
alternatives --set python /usr/bin/python3
dedticated non root user account + ssh keys copied from control node + sudo configured

++++++++++++++++++++++++++++++++1.5 Verifying Ansible Installation++++++++++++++++++++++++++++++++

ON CONTROL NODE USING PYTHON PIP INSTALLER (PLATFORM INDEPENDINT)

yum install python3
yum install python3-pip
alternatives --set python /usr/bin/python3
su - ansible
pip3 install ansible --user
ansible --version

ON CONTROL NODE USING SUBSCRIPTION MANAGER

subscription-manager repo --list
subscription-manager repo --enable=ansible-2-for-rhel-8-x86_64-rpms
yum install ansible
ansible --version
yum install python3

ON CONTROL NODE USING EPEL (USEFUL FOR CENTOS AND RHEL)

yum install epel-release
yum install ansible python3
ansible --version

ADD USER ON ALL NODES

useradd ansible
passwd ansible
echo "ansible ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/ansible

SSH KEY TRANSFER FROM CONRTOL NODE

su -ansible
ssh-keygen
ssh-copy-id node1;ssh-copy-id node2;

++++++++++++++++++++++++++++++++Lesson 1 Lab: Installing Ansible++++++++++++++++++++++++++++++++

##########################################################################################################################
LESSON #2 - SETTING UP MANAGED ENVOIRNMENT
##########################################################################################################################


++++++++++++++++++++++++++++++++2.1 Setting up Static Inventory++++++++++++++++++++++++++++++++
FILE LOCATION -
presnt in ansible.cfg file or old path is /etc/ansible/hosts
-i inventory option in case of multiple inventory files

EXAMPLE
[webservers]
web1.example.com
web2.example.com
192.0.2.42

[db-servers]
db1.example.com
db2.example.com

[servers:children]
```

```
webservers
db-servers

[east-datacenter]
web1.example.com
db1.example.com

[west-datacenter]
web2.example.com
db2.example.com

[production]
web1.example.com
web2.example.com
db1.example.com
db2.example.com

[development]
192.0.2.42

[usa]
washington[1:2].example.com

[canada]
ontario[01:02].example.com


[user@controlnode ~]$ ansible canada --list-hosts
  hosts (2):
    ontario01.example.com
    ontario02.example.com



++++++++++++++++++++++++++++++++++2.2 Understanding Dynamic Inventory++++++++++++++++++++++++++++++++++

use clould script to pull inventory

++++++++++++++++++++++++++++++++++2.3 Understanding Ansible Configuration Files++++++++++++++++++++++++++++++++++
#default                /etc/ansible/ansible.cfg
#this will overwrite default        ~/.ansible.cfg
#highest priority           ./ansible.cfg

-v FLAG TO CHECK CFG FILE IN USE
ansible servers --list-hosts -v


++++++++++++++++++++++++++++++++++2.4 Managing Ansible Configuration Files++++++++++++++++++++++++++++++++++
vi ansible.cfg

[defaults]
inventory = ./inventory
remote_user = user
ask_pass = false

[privilege_escalation]
become = true
become_method = sudo
become_user = root
become_ask_pass = false



ssh-keygen
ssh-copy-id root@server.example.com

```

```
automate ssh key copy on all managed hosts in inventory file
ssh-keygen
vi playbook.yaml

- name: Public key is deployed to managed hosts for Ansible
  hosts: all

  tasks:
    - name: Ensure key is in root's ~/.ssh/authorized_hosts
      authorized_key:
        user: root
        state: present
        key: '{{ item }}'
      with_file:
        - ~/.ssh/id_rsa.pub


put all of those managed hosts in group windows, and then create a file named group_vars/windows containing the following lines:

ansible_connection: winrm
ansible_port: 5986




++++++++++++++++++++++++++++++++++Lesson 2 Lab: Setting up an Ansible Managed Environment++++++++++++++++++++++++++++++++++

##########################################################################################################################
LESSON #3 -  ADHOC COMMANDS
##########################################################################################################################
++++++++++++++++++++++++++++++++++3.1 Using Ad Hoc Commands++++++++++++++++++++++++++++++++++
ansible host-pattern -m module [-a 'module arguments'] [-i inventory]

ansible all -m ping
ansible-doc ping or  ansible-doc -l(list all modules)

ansible-doc -l
ansible-doc modulename
ansible-doc user

Connecting to a host and install python using ansible
ansible nodename.example.com -u root --ask-pass -i inventory -m raw -a 'yum install python3'


++++++++++++++++++++++++++++++++++3.2 Understanding Ansible Modules++++++++++++++++++++++++++++++++++
Files modules

    copy: Copy a local file to the managed host

    file: Set permissions and other properties of files

    lineinfile: Ensure a particular line is or is not in a file

    synchronize: Synchronize content using rsync

Software package modules

    package: Manage packages using autodetected package manager native to the operating system

    yum: Manage packages using the YUM package manager

```

```
    apt: Manage packages using the APT package manager

    dnf: Manage packages using the DNF package manager

    gem: Manage Ruby gems

    pip: Manage Python packages from PyPI

System modules

    firewalld: Manage arbitrary ports and services using firewalld

    reboot: Reboot a machine

    service: Manage services

    user: Add, remove, and manage user accounts

Net Tools modules

    get_url: Download files over HTTP, HTTPS, or FTP

    nmcli: Manage networking

    uri: Interact with web services


Example- user add
ansible -m user -a 'name=newbie uid=4000 state=present' \
> servera.lab.example.com


++++++++++++++++++++++++++++++++++3.3 Using ansible-doc to get Module Documentation++++++++++++++++++++++++++++++++++++
ansible-doc -l
ansible-doc file
ansible-doc service
++++++++++++++++++++++++++++++++++3.4 Introducing Essential Ansible Modules++++++++++++++++++++++++++++++++++


++++++++++++++++++++++++++++++++++Lesson 3 Lab: Using Ad Hoc Commands++++++++++++++++++++++++++++++++++


##########################################################################################################################################
LESSON #4 -  GETTING STARTED WITH PLAYBOOKS
##########################################################################################################################################


++++++++++++++++++++++++++++++++++4.1 Using YAML to Write Playbooks++++++++++++++++++++++++++++++++++
vi $HOME/.vimrc
autocmd FileType yaml setlocal ai ts=2 sw=2 et
set nu

sdv_linux_test.yml
---
- name: this is a test playbook
  hosts: seint11,seint123
  tasks:
  - name: call moudule ping
    ping:
      arg1:
      arg2:

```

```
- name:
  task:
  - name:
    module:
      arg1:
      arg2:
  - name:
    module:
      arg1:
      arg2:




---
- name: Configure important user consistently
  hosts: servera.lab.example.com
  tasks:
    - name: newbie exists with UID 4000
      user:
        name: newbie
        uid: 4000
        state: present


Run Playbook
#ansible-playbook site.yml

Syntax check
#ansible-playbook --syntax-check webserver.yml

Executing a Check or Dry Run
ansible-playbook -C webserver.yml



++++++++++++++++++++++++++++++++++4.2 Verifying Playbook Syntax++++++++++++++++++++++++++++++++++
Syntax check
#ansible-playbook --syntax-check webserver.yml

Executing a Check or Dry Run
ansible-playbook -C webserver.yml

++++++++++++++++++++++++++++++++++4.3 Writing Multiple-Play Playbooks++++++++++++++++++++++++++++++++++
tasks example
tasks:
    - name: web server is enabled
      service:
        name: httpd
        enabled: true

    - name: NTP server is enabled
      service:
        name: chronyd
        enabled: true

    - name: Postfix is enabled
      service:
        name: postfix
        enabled: true


++++++++++++++++++++++++++++++++++Lesson 4 Lab: Getting Started with Playbooks++++++++++++++++++++++++++++++++++
lab playbook-basic start
cd /home/student/playbook-basic
```

```
cat ansible.cfg
cat inventory
vim  site.yml
---
- name: install and start httpd
  hosts: web
  tasks:
    - name: httpd package is present
      yum:
        name: httpd
        state: present

    - name: prepare index.html
      copy:
        src: files/index.html
        dest: /var/www/html/index.html

    - name: start httpd
      service:
        name: httpd
        state: started
        enabled: true
      register: output
    - name: print output
      debug::
        var: "{{output}}"

ansible-playbook --syntax-check site.yml
ansible-playbook site.yml
curl serverc.lab.example.com
curl serverd.lab.example.com




---
- name: Enable intranet services
  hosts: servera.lab.example.com
  become: yes
  tasks:
    - name: latest version of httpd and firewalld installed
      yum:
        name:
          - httpd
          - firewalld
        state: latest

    - name: test html page is installed
      copy:
        content: "Welcome to the example.com intranet!\n"
        dest: /var/www/html/index.html

    - name: firewalld enabled and running
      service:
        name: firewalld
        enabled: true
        state: started

    - name: firewalld permits access to httpd service
      firewalld:
        service: http
        permanent: true
        state: enabled
        immediate: yes

    - name: httpd enabled and running
      service:
        name: httpd
```

```
        enabled: true
        state: started

- name: Test intranet web server
  hosts: localhost
  become: no
  tasks:
    - name: connect to intranet web server
      uri:
        url: http://servera.lab.example.com
        return_content: yes
        status_code: 200


ansible-playbook --syntax-check intranet.yml
ansible-playbook -v intranet.yml
lab playbook-multi finish



install httpd package on test01
start httpd and firewall should allow http access
create /var/www/html/index.html with "hello world"

---
  - name: configure web services
    hosts: test01
    tasks:
      - name: install httpd package on test01
        yum:
          name: httpd
          state: latest

      - name: start firewalld
        service:
          name: firewalld
          enabled: true
          state: started

      - name: start httpd
        service:
          name: httpd
          enabled: true
          state: started

      - name: allow http access
        firewalld:
          service: http
          permanent: true
          state: enabled
          immediate: true

      - name: create /var/www/html/index.html with hello world text
        copy:
          content: "Welcome to the test01 intranet! \n"
          dest: /var/www/html/index.html


  - name: test web configuration
    hosts: localhost
    become: no
    tasks:
      - name: connect to the web server
        uri:
          url: http://test01
          return_content: true
          status_code: 200


```

```
Undo changes : remove web server, index.html  and disable firewall
---
  - name: remove web server, index.html  and disable firewall
    hosts: test01
    tasks:
      - name: remove web server
        package:
          name: httpd
          state: absent
      - name: remove index.html
        file:
          name: /var/www/html/index.html
          state: absent
      - name: remove http service from firewall
        firewalld:
          service: http
          state: disabled
          permanent: yes
      - name: test http://test01
        uri:
          url: http://test01
          return_content: yes
          status_code: 200



##########################################################################################################
#########################################
LESSON #5 -  WORKING WITH FACTS AND VARIABLES
##########################################################################################################
#########################################

++++++++++++++++++++++++++++++++++5.1 Understanding Variables++++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++5.2 Using Variables++++++++++++++++++++++++++++++++++++
test.yml
- hosts: all
  vars:
    user: joe
    home: /home/joe

or

test.yml
- hosts: all
  vars_files:
    vars/users.yml

use
{{variable name}}


or - highest scope
ansible-playbook site.yml -e "user=lisa"


++++++++++++++++++++++++++++++++++5.3 Understanding Variable Precedence++++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++5.4 Managing Host Variables++++++++++++++++++++++++++++++++++++
inventory file

[web_servers]
test01 web_package=httpd

or

[web_servers]
test01
```

```
test02
[web_servers:vars]
web_package=httpd


include files
~/project/host_vars/test01
or
~/project/group_vars/web_servers



#multi value / array
user:
  lisa:
    - username: lisa
      homedir: /home/lisa
      shell: /bin/bash
  linda:
    - username: linda
      homedir: /home/linda
      shell: /bin/bash
  anna:
    - username: anna
      homedir: /home/anna
      shell: /bin/bash

vars_files:
  vars/user_list
tasks:
  -name : print array
    debug:
      msg:"user {{user.username}} has home dir {{user.homedir}} and shell {{user.shell}}"
    loop: {{user}}



++++++++++++++++++++++++++++++++++5.5 Using Multi-valued
Variables++++++++++++++++++++++++++++++++++
#dictionary (with_item can not be used)
user:
  lisa:
    username: lisa
    homedir: /home/lisa
    shell: /bin/bash
  linda:
    username: linda
    homedir: /home/linda
    shell: /bin/bash
  anna:
    username: anna
    homedir: /home/anna
    shell: /bin/bash


vars_files:
  vars/user_dict
tasks:
  -name : print array
    debug:
      msg:"msg:"user {{user.lisa.username}} has home dir {{user.lisa.homedir}} and
      shell {{user.lisa.shell}}""



++++++++++++++++++++++++++++++++++5.6 Using Ansible
Vault++++++++++++++++++++++++++++++++++
---
  - name: create a user
```

```
    vars_files:
      secret.yml
    tasks:
      - name: add user
        user:
          name: {{user}}
          password: {{pass}}


ansible-vault create secret.yml
user: lisa
pass: redhat
:wq

ansible-playbook --ask--vault-pass create.yml

or

echo redhat > vault-pass
ansible-playbook --vault-password-file=vault-pass create.yml



Creating an Encrypted File
#ansible-vault create secret.yml
New Vault password: redhat
Confirm New Vault password: redhat

or
ansible-vault create --vault-password-file=vault-pass secret.yml

view
ansible-vault view secret1.yml

Editing an Existing Encrypted File
ansible-vault edit secret.yml
Vault password: redhat


Encrypting an Existing File
ansible-vault encrypt secret1.yml secret2.yml
New Vault password: redhat
Confirm New Vault password: redhat
Encryption successful

Decrypting an Existing File
ansible-vault decrypt secret1.yml --output=secret1-decrypted.yml
Vault password: redhat
Decryption successful

Changing the Password of an Encrypted File
ansible-vault rekey secret.yml
Vault password: redhat
New Vault password: RedHat
Confirm New Vault password: RedHat
Rekey successful


ansible-vault rekey \
> --new-vault-password-file=NEW_VAULT_PASSWORD_FILE secret.yml


ansible-playbook --vault-id @prompt site.yml
Vault password (default): redhat

ansible-playbook --vault-password-file=vault-pw-file site.yml
```

```
++++++++++++++++++++++++++++++++++5.7 Working with Facts++++++++++++++++++++++++++++++++++
ansible_facts varable

---
- name: show facts
  hosts: test01
  tasks:
    - name: print facts
      debug:
        var: ansible_facts

ansible-playbook facts/facts.yml

Specific fact
---
- name: show facts
  hosts: all
  tasks:
    - name: print ip address of all
      debug:
        msg: {{ansible_facts.default_ipv4.address}}

ansible-playbook facts/ipfact.yml

---

gather_facts: no


++++++++++++++++++++++++++++++++++5.8 Creating Custom Facts++++++++++++++++++++++++++++++++++
dynamic facts stored in /etc/ansible/facts.d/filename.fact
store the facts in the following format
[customfacts]
package = samba
service = smbd
state = started
retrieve using ansible_facts.ansible_local

display localfacts - ansible hostname -m setup -a "filter=ansible_local"

set facts on managed hosts using playbook

---
- name: install custom facts
  hosts: lamp
  vars:
    remote_dir: /etc/ansible/facts.d
    facts_file:localfacts.fact
  tasks:
    - name: create fact directory
      file:
    path: {{remote_dir}}
    state: directory
        recurse: true

    - name: install new facts
      copy:
        src: {{facts_file}}
        dest:




Capturing Command Output with Registered Variables with register

- name: Installs a package and prints the result
  hosts: all
```

```
  tasks:
    - name: Install the package
      yum:
        name: httpd
        state: installed
      register: install_result

    - debug: var=install_result


++++++++++++++++++++++++++++++++Lesson 5 Lab: Working with Variables and Facts++++++++++++++++++++++++++++++++

create a playbook that will first define localfacts on the file servers,
these facts should define the name of the smbd service and samba package
ensure first play copies these facts to the file server
in the same playbook use a second play that will install and enable the smbd service


---
- name: define localfacts on the test01, install samba and start smbd
  hosts: test
  vars:
    remote_dir: /etc/ansible/facts.d
    fact_file: localfacts.fact
  tasks:
    - name: create facts directory on test01
      file:
        path: "{{remote_dir}}"
        state: directory
        recurse: yes

    - name: copy facts to test01
      copy:
        src: "{{ fact_file }}"
        dest: "{{ remote_dir }}"
- name: use a second play that will install and enable the smbd service
  hosts: test
  tasks:
    - name: install package as per custom fact
      package:
        name: "{{ ansible_facts.ansible_local.localfacts.localfacts.package }}"
        state: latest

    - name: start service as per custom fact
      service:
        name: "{{ ansible_facts.ansible_local.localfacts.localfacts.service  }}"
        enabled: true
        state: "{{ ansible_facts.ansible_local.localfacts.localfacts.state  }}"




---
- name: undo samba install remove facts file
  hosts: test
  vars:
    remote_file: /etc/ansible/facts.d/localfacts.fact
  tasks:
    - name: remove package as per custom fact
      package:
        name: "{{ ansible_facts.ansible_local.localfacts.localfacts.package }}"
        state: absent

    - name: remove custom facts file
      file:
        path: "{{remote_file}}"
        state: absent

```

```
851
852
853
854
855   ##########################################################################################
      ########################################
856   LESSON #6 -  USING TASK CONTROL
857   ##########################################################################################
      ########################################
858
859   ++++++++++++++++++++++++++++++++6.1 Using Loops and
      Items++++++++++++++++++++++++++++++++
860   #ARRAY
861
862   - name: start some services
863     service:
864       name: "{{item}}"
865       state: restarted
866     loop:
867       - vsftpd
868       - httpd
869
870       OR
871
872   - name: start some services
873     vars:
874       my_services:
875           - vsftpd
876           - httpd
877     service:
878       name: "{{ item }}"
879       state: started
880     loop: "{{ my_services }}"
881
882
883   #DICTIONARY
884   - name: start some services
885     user:
886       name: "{{item.name}}"
887       group: "{{item.groups}}"
888       state: present
889     loop:
890       - name: name1
891         groups: group1
892       - name: name2
893         groups: group2
894
895   ++++++++++++++++++++++++++++++++6.2 Using Register Variables with
      Loops++++++++++++++++++++++++++++++++
896   - name: how register works
897     hosts: all
898     tasks:
899       - name: show loop register
900         shell: "echo {{ item }}"
901         loop:
902           - "One"
903           - "Two"
904         register: output
905
906       - name: show results
907         debug:
908           msg: "the output of the shell command is \n"
909           var: output
910
911
912   ++++++++++++++++++++++++++++++++6.3 Using when to Run Tasks
      Conditionally++++++++++++++++++++++++++++++++
913   when a operator b
914   when ansible_machine=="x86_64"
```

```
< > <= >= !=
when ansible_machine is defined
when ansible_machine is not defined
when ansible_machine
when ansible_distribution in supported_distors

-name:when demo
hosts: all
vars:
    supported_distors:
        -redhat
        -fedora
        -centos
tasks:
-name: install redhat family packages
    yum:
        name: httpd
        state: present
    when: ansible_distribution in supported_distors


when ansible_distribution == "RedHat" or ansible_distribution == "CentOS"
when ansible_machine == "x86_64" and ansible_distribution == "RedHat"

++++++++++++++++++++++++++++++++++6.4 Testing Multiple Conditions++++++++++++++++++++++++++++++++++
-name:restart sshd only if crond is running
hosts: all
tasks:
-name: check if crond is running
    command: /usr/bin/systemctl is-active crond
    ignore_erros:true
    register:result

-name: restart sshd if crond is runing
    service:
        name: sshd
        state: restarted
    when:result.stdout == "Active" or result.rc == 0



++++++++++++++++++++++++++++++++++6.5 Using Handlers++++++++++++++++++++++++++++++++++
-name:setup webserver and use handler to restart webserver
hosts:all
force_handlers:true
tasks
    -name:install http
    pakage:
    name:httpd
    state:latest

    -name:copy html
    copy:
    src:/tmp/index.html
    dest:/var/www/html/index.html
    notify:
    - restart_web
    -name: fail this task
    copy:
    src:/tmp/nothing.txt
    dest: /tmp/nothing.txt
handlers:
    -name: restart_web
    service:
        name:httpd
        state:restarted
```

```
++++++++++++++++++++++++++++++++++6.6 Using Blocks++++++++++++++++++++++++++++++++++
---
- name: setup blocks sample
hosts:all
tasks:
-name: setup http
 block:
 -name: install http
  yum:
   name: http
   state: latest
  -name:start http
   service:
   name:httpd
   state:started
 when:ansible_distribution == "RedHat"


---
- name: complete block structure
  hosts: all
  tasks:
    - name: define block
      block:
        - name: remove a file
          file:
            path: /var/www/html/index.html
            state: absent
      rescue:
       - name: if remove fails create a file
         file:
           path: /tmp/rescue.txt
           state: present
      always:
       - name: always perform this task
         shell:
           cmd: "/usr/bin/logger hello"


++++++++++++++++++++++++++++++++++6.7 Dealing with
Failures++++++++++++++++++++++++++++++++++
exit status is the deciding factor for failure.
ignore_error:true
force_handlers:true (this one because handlers do not run if any task fails)

failed_when - to recognize a failure
failure.yml
---
- name: demo failed_when
  hosts: all
 tasks:
  - name: run a script
    command: echo hello world
    ignore_errors: true
    register: command_result
    failed_when: "'world' in command_result.stdout"
  - name: see if we get there
    debug:
      msg: hello


fail module
failure2.yml
---
- name: demo fail module
  hosts: all
 tasks:
  - name: do something impossible
```

```
      lvol:
        lv: lvxyz
        vg: vgxyz
        size: 1G
      register:command_result
      ignore_errors: true

    - name: show command_result erros
      debug:
        var: command_result

    - name: Print nice error message on failure
      fail:
        msg: Volume Group Does Not Exist!
        when: "'not found' in command_result.err"


++++++++++++++++++++++++++++++++++6.8 Managing Changed Status++++++++++++++++++++++++++++++++++

changed_when
changed.yml
---
- name: demo changed status
  hosts: all
 tasks:
  - name: check localtime
    command: date
    register: command_result
    changed_when: false


  - name: print local time
    debug:
      msg: command_result.stdout


comment changed_when and rerun the playbook
++++++++++++++++++++++++++++++++++Lesson 6 Lab: Using Task Control++++++++++++++++++++++++++++++++++
install and run mariadb should also install python3-PyMySQL package on node using RHEL8 on successful start of databse service, use mysql_user module to set databse root user and password

task_control.yml
---
-name:install mariadb on all hosts and install python3-PyMySQL on node using RHEL8
hosts:all
vars:
db_packages:
- mariadb-server
extra_packages:
- python3-PyMySQL
tasks:
-name: install mariadb
yum:
name:"{{item}}"
state: latest
loop:"{{db_packages}}"
when: ( ansible_distribution == "RedHat" or ansible_distribution == "CentOS" )  and ansible_distribution_major == "7"

-name: install python3-mysql
yum:
name:"{{item}}"
state: latest
loop:"{{extra_packages}}"
when: ( ansible_distribution == "RedHat" or ansible_distribution == "CentOS" )  and ansible_distribution_major == "8"
```

```
-name:on sucessful start of db service, use mysql_user module to ser database root user's password
service:
name:mariadb
state: started
enabled:true
notify:
- set db password

handlers:
-name: set db password
mysql_user:
name: root
password: password




##########################################################################################################################
LESSON #7 -  DEPLOYING FILES
##########################################################################################################################
++++++++++++++++++++++++++++++++++7.1 Using Modules to Manipulate Files++++++++++++++++++++++++++++++++++
lineinfile
blockinfile
fetch
file

file.yml
---
-name:create a file
hosts:test
tasks:
file:
path:/tmp/removeme
owner:ansible
mode:0640
state:touch
setype: public_context_rw_t



copy
copy.yml
---
-name:demo copy module
hosts:test
tasks:
-name: copy file demo
copy:
src: /etc/hosts
dest: /tmp/

-name: add some lines to  /tmp/hosts
blockinfile:
block: |
192.168.4.110 host1.example.com
192.168.4.120 host2.example.com
state:present

-name: verify file checksum
stat:
path:/tmp/hosts
checksum_algorithm:md5
register:result
-debug
```

```
msg:"the checksum of /tmp/hosts is {{ result.stat.checksum  }}"

-name:fetch a file
fetch:
src: /tmp/hosts
dest:/tmp/

++++++++++++++++++++++++++++++++++7.2 Managing SELinux File Context++++++++++++++++++++++++++++++++++
file: manages file and selinux context
sefcontext:manages selinux file context
    requires - policoreutils-python-utils


---
-name:demo sefcontect
hosts:test
tasks
-name:install all required packages
yum:
name: policoreutils-python-utils
state:latest
-name:set selinux context
sefcontext:
target:/tmp/removeme
setype:tmp_t
state:present
notify:
-run restorecon

handler:
-name: run restorecon
command: restorecon -v /tmp/removeme



++++++++++++++++++++++++++++++++++7.3 Using Jinja2 Templates++++++++++++++++++++++++++++++++++
mkdir templates;cd templates
vi vsftpd.j2

anonymous_enable={{ anonymous_enable }}
local_enable={{ local_enable }}
write_enable={{ write_enable }}
anon-upload_enable={{anon_upload_enable }}
dirmessage_enable=YES
xferlog_enable=YES
connect_from_port_20=YES
pam_service_name=vsftpd
user_list_enable=YES
#my_ip_address={{anisble_facts['default_ipv4']['address']}}


vim vsftpd_template.yml
---
-name: jinja2 template demo
hosts: test01
vars:
anonymous_enable: yes
local_enable: yes
write_enable: yes
anon_upload_enable: yes

tasks:
-name:install vsftpd
yum:
name:vsftpd
state:latest
-name:use j2 template to copy vsftpd config
```

```
template:
src:vsftpd.j2
dest:/etc/vstpfd/vsftpd.conf

++++++++++++++++++++++++++++++++++7.4 Using Control Structures in Jinja2++++++++++++++++++++++++++++++++++

vim /templates/hosts.j2

{% for hosts in group['all'] %}
{{ hostvars [host]['ansible_facts']['default_ipv4']['address'] }} {{ hostvars [host]['ansible_facts']['fqdn'] }} {{hostvars [host]['ansible_facts']['hostname'] }}
{% endfor %}

++++++++++++++++++++++++++++++++++Lesson 7 Lab: Deploying Files with Templates++++++++++++++++++++++++++++++++++
configure anonymous ftp upload with the following settings
vsftpd.conf is modified to allow anonymous ftp access and uploads
the /var/ftp/pub is configured with appropriate permissions and selinux context
the selinux boolean ftpd_anon_write is set to on

create a playbook that ensures that the vsftpd  serivice is insatlled and enabled
the firewall is opened and the above requirements are met
define variables in the playbook to set vsftpd.conf parameters and use these in the template

mkdir templates;cd templates
vi vsftpd.j2

anonymous_enable={{ anonymous_enable }}
local_enable={{ local_enable }}
write_enable={{ write_enable }}
anon-upload_enable={{anon_upload_enable }}
dirmessage_enable=YES
xferlog_enable=YES
connect_from_port_20=YES
pam_service_name=vsftpd
user_list_enable=YES
#my_ip_address={{anisble_facts['default_ipv4']['address']}}




vim vsftpd_j2_lab.yml
---
- name: install vsftpd and modify vsftpd.conf using j2
  hosts: test
  ignore_errors: yes
  vars:
    anonymous_enable: yes
    anon_upload_enable: yes
    local_enable: yes
    write_enable: yes
  tasks:
    - name: install policycoreutils-pyhton-utils and vsftpd as per specifications
      yum:
        name: "{{ item }}"
        state:  present
      loop:
        - libsemanage-python
        - vsftpd
        - policycoreutils-python
    - name: start and enable vsftpd and firewalld
      service:
        name: "{{item}}"
        state: started
        enabled: true
      loop:
        - vsftpd
```

```
          - firewalld
      - name: enable firewall for ftp
        firewalld:
          service:  ftp
          state:  enabled
          permanent: true
          immediate:  true

      - name: use j2 template to copy vsftpd config
        template:
          src:  vsftpd.j2
          dest: /etc/vstpfd/vsftpd.conf

      - name: set permissions on /var/ftp/pub
        file:
          path: /var/ftp/pub
          mode: '0777'

      - name: set selinux bool
        seboolean:
          name: ftpd_anon_write
          state: yes
          persistent: yes

      - name: set selinux context  on /var/ftp/pub
        sefcontext:
          target: /var/ftp/pub
          setype: public_context_rw_t
          state:  present
        notify:
          - run restorecon

      - name: run restorecon
        command: restorecon -vR /var/ftp/pub


  ---
  -name: test vsftpd
  host:localhost
  tasks:
  -name:install lftp
  yum:
  package:lftp
  state:present
  -name:upload test file
  shell: >
  lftp test01 <<EOF
  cd pub
  put /etc/demo
  EOF

  ##########################################################################################
  ########################################
  LESSON #8 -  ANSIBLE ROLES
  ##########################################################################################
  ########################################
  ++++++++++++++++++++++++++++++++8.1 Understanding Directory Structure Best
  Practices++++++++++++++++++++++++++++++++++
  ~/myproject/ansible.cfg
      inventory
      inventory-dev
      inventory-prod
      site.yml -> main config file which is small and mainly have includes like lamp.yml,
      file.yml,
      group-vars
      host-vars

  directory struture of role
  myrole/
```

```
    defaults/
        main.yml # default variables for a play and will be overwritten with play level
        variables

    files/ #include files required by the play

    handlers/
        main.yml
    meta/
        main.yml #includes info about author, license, dependencies etc

    README.md

    tasks/
        main.yml # all tasks are here

    templates/ # store jinja2 templates

    tests/ # can be used to test the role
        inventory
        test.yml
    var/
        main.yml # standard variables which are NOT to be overwriten by play

++++++++++++++++++++++++++++++++++8.2 Understanding Ansible Roles++++++++++++++++++++++++++++++++++
rhel-system-roles
community provided

roles precedence
./roles
~/.ansible/roles
/etc/ansible/roles
/usr/share/ansible/roles


roles execute before any task in a playbook
---
-name:
hosts:all
roles:
- role1
- role2
  var1: some value
  var2: some value

++++++++++++++++++++++++++++++++++8.3 Using Ansible Galaxy for Standard Roles++++++++++++++++++++++++++++++++++
galaxy.ansible.com


ansible-galaxy install geerlingguy.nginx
install location will be cd ~/.ansible/roles/

#Use this role
vim lesson8/nginx-role.yml

---
-name:use galaxy nginx role
hosts: test01
user:ansible
become; true
roles:
    -role: geerlingguy.nginx

ansible-playbook nginx-role.yml

++++++++++++++++++++++++++++++++++8.4 Using the Ansible Galaxy Command Line Tool++++++++++++++++++++++++++++++++++
```

```
ansible-galaxy search
ansible-galaxy search  'wordpres' --platforms EL --author bertvv
ansible-galaxy info bertvv.wordpress
ansible-galaxy install bertvv.wordpress
ansible-galaxy list #will list installed roles
ansible-galaxy remove
ansible-galaxy init user.myrole

#
roles/requirements.yml

- src:location/myrole.tar
name: myrole
version: 1.0

#install galaxy role requirements
ansible-galaxy install -r roles/requirements.yml

#demo requirements.yml
vim role/requirements.yml

#from ansible galaxy
- src: geerlingguy.docker
- src: geerlingguy.docker
  version: 2.7.0
  name: nginx


ansible-galaxy install -r roles/requirements.yml -p roles


++++++++++++++++++++++++++++++++++8.5 Creating Custom Roles++++++++++++++++++++++++++++++++++
ansible-galaxy init myrole
#put it in ~/.ansible/roles
#should have version contol repo
# edit README.md and meta/mail.yml , as it should contain dependencies and documentation about the role
# one role per task, multiple roles for multiple tasks

#Conditional roles demo
---
-name:
host:test01
tasks:
-include-role:
    name: lamp
when: "ansible_facts.os_family == 'RedHat'"

#role demo
cd lesson8
ansible-galaxy init motd

vi tasks/mail.yml
#tasks file for motd
- name: copy motd file
  templates:
   src: templates/motd.j2
   dest:/etc/motd
   owner:root
   group:root
   mode:0444

vi templates/motd.j2
welcome to {{ ansible_hostname }}
this file was created on {{ ansible_date_time.date }}
go away of you have no business
contact {{ system_manager }} if anything goes wrong
```

```
vi defaults/mail.yml
#defaults for motd
system_manager: amit@example.com

vim meta/mail.yml

#add your name
remove directories which do not have purpose like handlers and vars if not in use

write a playbook to use motd role
vim lesson8/motd-role.yml
---
-name: use motd role
hosts: test
user: ansible
become:true
roles:
 - role:motd
   system_manager: ab@gmail.com

hosts:test01


++++++++++++++++++++++++++++++++++8.6 Managing Order of Execution++++++++++++++++++++++++++++++++++

pre_task - run task before role

vim pretask.yml

---
-name:run task before role
hosts:test
pre_tasks:
-name: update yum cache
yum:
update_cache:yes
roles:
- motd
post task - run after playbook roles and tasks

++++++++++++++++++++++++++++++++++8.7 Understanding RHEL System Roles++++++++++++++++++++++++++++++++++

++++++++++++++++++++++++++++++++++Lesson 8 Lab: Using Ansible Roles++++++++++++++++++++++++++++++++++

create a role that configures an apache vhost on test group
use tempalate to use vhost config , all refernces to hostname are replaced with ansible facts
use adhoc command to test working of virtual host

cd /lesson8/roles/
ansible-galaxy init vhost

vim templates/vhost.conf.j2

<VirtualHost *:80>
    ServerAdmin webmaster@{{ansible_fqdn}}
    ErrorLog logs/{{ansible_hostname}}_error.log
    Customlog logs/{{ansible_hostname}}_common.log common
    DocumentRoot /var/www/html/{{ansible_hostname}}/
    ServerName {{ansible_facts.fqdn}}
    <Directory>
    Options +Indexes +FollowSymlinks +Includes
    Order allow,deny
    Allow from all
    </Directory>
</VirtualHost>
```

```
vim templates/index.j2
<h3>HELLO FROM SERVER {{ansible_hostname}}</h3>

vim tasks/main.yml
-name: install apache
yum:
name:httpd
state:latest

-name: start service httpd
service:
name: httpd
state: started
enabled: true

-name:copy vhostfile
template:
src: templates/vhost.conf.j2
dest: /etc/httpd/conf.d/vhost.conf
owner:root
group:root
mode:0644
notify:
 - restart_httpd

-name:configure vhost directories
file:
state:directory
path:/var/www/html/{{ansible_hostname}}
mode:0755

-name: copy index
template:
src:templates/index.j2
dest:/var/www/html/{{anisble_hostname}}/index.html


vim handlers/main.yml
-name: restart_httpd
service:
name:httpd
state:restarted

vim apache-vhost-role.yml
---
-name apache vhost lab
hosts:test
roles:
 - vhost


#test
ansible localhost -m command -a "curl -v http://test01/"

Git REPO
https://github.com/abhatt014/role-vhost.git

steps
cd lesson8
#initial git config

git config --global user.email amitfreeze@gmail.com
git config --global user.name abhatt014
git config --global credential.helper cache
git config --global credential.helper 'cache --timeout=36000'
git init
```

```
git status
git add .
git commit -m "log message"
git remote add origin https://github.com/abhatt014/role-vhost.git
git push -u origin master



git pull origin master
git status
git add .
git commit -m "log message"
git push -u origin master

##########################################################################################
########################################
LESSON #9 -  RHEL SYSTEM ROLES
##########################################################################################
########################################
++++++++++++++++++++++++++++++++++9.1 Understanding RHEL System
Roles++++++++++++++++++++++++++++++++++
present from rhel7.4 and for rhel6.10 and later
used to configure standard rhel operations
makes easy to manage different versions of rhel
install rhel-system-roles package

current systemroles
rhel-system-roles-kdump
rhel-system-roles-network
rhel-system-roles-selinux
rhel-system-roles-timesync
rhel-system-roles-postfix
rhel-system-roles-firewall
rhel-system-roles-tuned


++++++++++++++++++++++++++++++++++9.2 Installing RHEL System
Roles++++++++++++++++++++++++++++++++++

yum install rhel-system-roles
path - /usr/share/ansible/roles
view yaml files in roles dir

tree /usr/share/ansible/roles/<any role>
++++++++++++++++++++++++++++++++++9.3 Using the RHEL SELinux System
Role++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++9.4 Using the RHEL TimeSync System
Role++++++++++++++++++++++++++++++++++

checkout example at /usr/share/doc/rhel-system-roles/timesync
cd lesson9
vim setuptime.yml

---
-name:setting up time
hosts:all
#use vars from example at /usr/share/doc/rhel-system-roles/timesync
vars:
- hostname: 0.pool.ntp.org
iburst: yes
- hostname: 1.pool.ntp.org
iburst: yes
- hostname: 2.pool.ntp.org
iburst: yes
- hostname: 3.pool.ntp.org
iburst: yes
timezone: Asia/Kolkata
roles:
-rhel-system-roles.timesync
```

```
tasks:
-name: set timezone
timezone:
    name:"{{timezone}}"

==================lite version with varialbes in group vars dir
vim group_vars/servers/timesync.yml

timesync_ntp_servers:
- hostname: 0.pool.ntp.org
iburst: yes
- hostname: 1.pool.ntp.org
iburst: yes
- hostname: 2.pool.ntp.org
iburst: yes
- hostname: 3.pool.ntp.org
iburst: yes
timezone: Asia/Kolkata


vim setuptime-lite.yml
---
-name:setting up time
hosts:all
roles:
-rhel-system-roles.timesync
tasks:
-name: set timezone
timezone:
    name:"{{timezone}}"




++++++++++++++++++++++++++++++++++Lesson 9 Lab: Using RHEL System Roles++++++++++++++++++++++++++++++++++
use a rhel-system-role to setup network cards in managed machines and use dhcp
vim rhel-system-role-network-lab.yml

--
- hosts: test
  vars:
    network_connections:
      - name: prod1
        state: up
        type: ethernet
        autoconnect: yes
        mac: 00:00:00:00:00:00
        mtu: 1450

  roles:
    - rhel-system-roles.network


##########################################################################################################################################
LESSON #10 -  ANSIBLE IN LARGE ENVOIRNMENTS
##########################################################################################################################################
++++++++++++++++++++++++++++++++++10.1 Managing Inventory++++++++++++++++++++++++++++++++++
ansible-inventory --list
++++++++++++++++++++++++++++++++++10.2 Addressing Host Patterns++++++++++++++++++++++++++++++++++
-host:'*.example.com'
-host:'192.168.*'
-host:'web*'
-host:db1,db2,192.168.4.2
```

```
-host:web,!web1

++++++++++++++++++++++++++++++++++10.3 Configuring Parallelism++++++++++++++++++++++++++++++++++
processing order
forks = n # to change parallism
or ansible-playbook name.yml -f 4 # parallism is 4

Rolling Updates - use serial keyword to run tasks in batches of server
++++++++++++++++++++++++++++++++++10.4 Organizing Directory Structure++++++++++++++++++++++++++++++++++

cd lesson10
vim includes.yml
---
- name setup apache
hosts: test
vars:
package:httpd
service:httpd
tasks:
-name: setup apache task
import_tasks: install-and-setup.yml

vim install-and-setup.yml
-name:
yum:
name:"{{pacakge}}"
state:present
-name:
service:
name:"{{service}}"
state:started
enabled:true
++++++++++++++++++++++++++++++++++Lesson 10 Lab: Using Ansible in Large Environments++++++++++++++++++++++++++++++++++
git clone https://github.com/sandervanvugt/rhce8

clone this and fix it

##########################################################################################################################################
LESSON #11 -  TROUBLESHOORING GUIDE
##########################################################################################################################################
++++++++++++++++++++++++++++++++++11.1 Managing Ansible Logs++++++++++++++++++++++++++++++++++
in ansible.cfg
log_path=~/projectdir
++++++++++++++++++++++++++++++++++11.2 Using the Debug Module++++++++++++++++++++++++++++++++++
debug:
msg:
or
var:
++++++++++++++++++++++++++++++++++11.3 Using Check Mode++++++++++++++++++++++++++++++++++
check mode - C
ansible-playbook -C play.yml --diff # if using jinja tempaltes
++++++++++++++++++++++++++++++++++11.4 Using Modules for Troubleshooting and Testing++++++++++++++++++++++++++++++++++
uri -
script - run a script on managehosts
stat - check status of file, if file exists, register a variable
assert - this module will fail if certain conditions are not met

++++++++++++++++++++++++++++++++++11.5 Troubleshooting Connection Issues++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++11.6 Analyzing Playbook
```

Errors++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++11.7 Avoiding Errors in Playbook Best Practices++++++++++++++++++++++++++++++++++

++++++++++++++++++++++++++++++++Lesson 11 Lab: Troubleshooting Ansible++++++++++++++++++++++++++++++++++

##
LESSON #12 - MANAGEING SOFTWARE
##
++++++++++++++++++++++++++++++++12.1 Understanding Modules Related to Software Management++++++++++++++++++++++++++++++++++
Modules:

package
win_package
apt , yum
yum_repository
package_facts - package facts
rpm_key - for gpg keys
redhat_subscription - manage subscriptions
rhn_register - resiger the system to rhn
rhn_channel - manage rhn channel

++++++++++++++++++++++++++++++++12.2 Implementing a Playbook to Manage Software++++++++++++++++++++++++++++++++++

demo mange software

vim package-manage.yml

-name:
hosts:test
vars:
my_package: nmap
tasks:

-name:create a repo
yum_repository:
name:lessn12
desc:lesson12
file:lesson12
baseurl:ftp://control.example.com/repo
gpgcheck:0

-name: install apackage
yum:
name:"{{my_package}}"
state:present

-name: get package facts
package_facts:
manager:auto

-name: show package facts for my_package
debug:
var:package_facts.packages.my_package
when: my_package in ansible_facts.packages

++++++++++++++++++++++++++++++++Lesson 12 Lab: Managing Software++++++++++++++++++++++++++++++++++
setup a repo on control.example.com, this repo should offer multiple files including nmap
provide a file list using varables
configure ansible1 to use the newlycreated repo , install nmap from this repo

```
vim lesson12-config-repo.yml
---
#play1
-name: lesson12 lab
hosts: localhost
tasks:
-name: install ftp
yum:
name: vsftpd
state: present

-name: start ftp
service:
name:vsftpd
state:started
enabled:true

-name: open firewall for ftp
firewalld:
name: ftp
state: enabled
premanent: true
immediate: true

-name: create a new repo directory
file:
state:directory
path:/var/ftp/repo

-name: install createrepo and yumdownloader using yum-utils
yum:
name:
- createrepo
-yum-utils
state:present


-name: download nmap in new repo
command: yumdownloader nmap --destdir /var/ftp/repo

-name:create a new repo for managed nodes
command: createrepo /var/ftp/repo


#play2
---
-name: confire managedhosts to use the new repo and install nmap
hosts:test
tasks:
-name:add new repo
yum:repository:
name:lab12
desc:lab12
file:lab12
baseurl:ftp://control.example.com/repo
gpgcheck:0

-name:install nmap
yum:
name:nmap
state:present

##########################################################################################
########################################
LESSON #13 -  MANAGEING USERS
##########################################################################################
########################################
++++++++++++++++++++++++++++++++13.1 Understanding Modules Related to User
Management++++++++++++++++++++++++++++++++
```

Modules -
user:
group:
pamd:
authorized_key: transfers keys from control to managed
lininfile:

++++++++++++++++++++++++++++++++++13.2 Implementing a Playbook to Manage Users++++++++++++++++++++++++++++++++++
var/users
users:
- username: linda
groups: sales
- username: lori
 groups: sales
- username: lisa
 groups: account
- username: lucy
 groups: account

var/groups
usergroups:
- groupname: sales
- groupname: accounts

vi demoprepare

- name: prepare dummy ssh keys for this demo
hosts: localhost
vars_files:
- vars/users
tasks:
-name: make file dir in /home/ansible/lesson13/
file:
path: /home/ansible/lesson13/files/{{item.username}}
state: directory
loop:"{{users}}"

-name: touch id_rsa.pub to all user's dir in file dir
file:
state:touch
path:/home/ansible/lesson13/files/{{item.username}}/id_rsa.pub
loop:"{{users}}

vi demousermangement

- name: create some users
hosts: test
var_files:
- vars/users
- var/groups
tasks:
- name: add groups
group:
name: "{{item.groupname}}"
loop: "{{usergroups}}"

- name: add users
user:
name: "{{item.username}}"
group: "{{item.groupname}}"
loop:"{{users}}"

```
- name: add ssh keys
authorized_keys:
user:"{{item}}"
file:"{{lookup('file','file/'+item.username+'/id_rsa.pub')}}"
loop: "{{users.username}}"

- name: add sales group members to sudo
copy:
content:"%sales ALL=(ALL) NOPASSWD: ALL"
dest:/etc/sudoers.d/sales

- name: disable root ssh login
lininfile:
dest:/etc/sshd/sshd_config
regex:"^PermitRootLogin"
line:"PermitRootLogin no"
notify:
- restart sshd

- name: restart sshd
service:
name:sshd
state:restarted
enabled:true




manage sudo


var/defaults
sudo_groups:
- name:developers
groupid:5000
sudo:false
- name:admins
groupid:5001
sudo:true
- name:dbas
groupid:5002
sudo:false

vi sudoers.j2

{% for items in sudo_groups %}
{% if item.sudo %}
{{item.group}} ALL=(ALL) NOPASSWD: ALL
{% endif %}
{% endfor %}

vi sudosetup
---
-name: sudo manage
hosts: test
vars_files:
- var/defaults
 - name: allow group members in sudo
template:
src: sudoers.j2
dest: /etc/sudoers.d/sudogroups
validate: 'visudo -cf %s'
mode: 0440



++++++++++++++++++++++++++++++++13.3 Managing Encrypted
Passwords++++++++++++++++++++++++++++++++
```

```
generate sha512 password hash
ansible localhost -m debug -a "msg={{'redhat'| password_hash('sha12','ransomsalt')}}"

use the has in user module
---
-name:
hosts: test
vars:
- password_hash="hash"
tasks:
-name:create user
user:anna
password:"{{password_hash}}"



++++++++++++++++++++++++++++++++Lesson 13 Lab: Managing Users++++++++++++++++++++++++++++++++

create users meeting the following requirements
    groups in the vars/defaults file should be created if they do not exist
    use variable file to define users and groups
        bill and bob are part of group admins
        bea and betty are part of group dev
        brenda and benny are part of group dbas
    do this way that ssh authorized keys are setup automatically


vim var/defaults
sudo_groups:
- name:dev
groupid:5000
sudo:false
- name:admins
groupid:5001
sudo:true
- name:dbas
groupid:5002
sudo:false

vim vars/users
users:
-username: bill
group: admins
-username: bob
group: admins
-username: bea
group: dev
-username: betty
group: dev
-username: brenda
group: dbas
-username: benny
group: dbas

#create a password
ansible localhost -m debug -a "msg={{'redhat' | password_hash('sha12','randomsalt')}}"


#setup sudoers.j2
{% for item in sudo_groups %}
{% if item.sudo %}
{{item.group}} ALL=(ALL) NOPASSWD: ALL
{% endif %}
{% endfor %}


vim lab13-part1
---
```

```
-name:lab lesson13 part1
vars_files:
- var/users
hosts:localhost
tasks:
-name: create sshkeys dir structure
file:
state:directory
path:/home/ansible/lesson13/{{item.username}}
loop:"{{users}}"

file:
state:touch
path:path:/home/ansible/lesson13/{{item.username}}/id_rsa.pub
mode: "0600"


vim vim lab13-part2
---
-name:lab lesson13 part2
hosts:test
vars:
- password: "hash"
vars_files:
- vars/defaults
- vars/users
tasks:

-name: create groups , use var/defaults
group:
name:"{{item.name}}"
state: present
loop:"{{sudo_groups}}"

-name: create users with password, use var/users
user:
name:"{{item.username}}"
group:"{{item.group}}"
password:"{{password}}"
loop:"{{users}}"

-name: set password ageing 0
command: chage -d 0 {{item.username}}
loop: "{{users}}"

-name: copy ssh keys in user directories
authorized_keys:
user:"{{item.username}}"
file:"{{lookup('file',file/'+{{item.username}}+'/id_rsa.pub')}}"
loop:"{{users}}"

-name: setup sudoers as per data provided in defaults var file
template:
src:template/sudoers.j2
dest:/etc/sudoers.d/sudouser
validate:'visudo -cf %s'
mode:"0400"



################################################################################################################################
LESSON #14 -  MANAGEING PROCESS AND TASKS
################################################################################################################################
++++++++++++++++++++++++++++++++++14.1 Understanding Modules for Managing Processes and Tasks++++++++++++++++++++++++++++++++++
cron:
at:
```

file:
service:
service_facts:
systemd:
++++++++++++++++++++++++++++++++14.2 Implementing a Playbook to Manage Processes and Tasks++++++++++++++++++++++++++++++++
vim demo-create-cron.yml

-name:
hosts:
tasks:
-name:
cron:
name:write message to file
minute:"*/2"
hour:8-18
user:ansible
job:echo "entry writen at $(date)" >> /tmp/cronlog
cron_file:state:cron_file
state:present

vim demo-delete-cron.yml

-name:
hosts:
tasks:
-name:
cron:
name:write message to file
cron_file:state:cron_file
user:ansible
state:absent

vim demo-at.yml

-name:
hosts:
tasks:
-name:
at:
command:"date > /tmp/my-at-file"
count :1
units :minutes
unique :yes
state:present

vim setup-boot-target

-name:
hosts:
tasks:
-name: setup boot target to graphical
file:
src:/usr/lib/systemd/system/graphical.target
dest:/etc/systemd/system/default.target
state:link

++++++++++++++++++++++++++++++++Lesson 14 Lab: Managing Processes and Tasks++++++++++++++++++++++++++++++++

```
vim lab14.yml
tasks:
-name:set the default boot state of the servers to multi-user.target
file:
src:/usr/lib/systemd/system/multi-user.target
dest:/etc/systemd/system/default.target
state:link

-name: reboot server
reboot:

-name: once the server is reboot show message: successfully rebooted
debug:
msg: "reboot success "


##########################################################################################################################################
LESSON #15 -  MANAGEING STORAGE
##########################################################################################################################################
++++++++++++++++++++++++++++++++++15.1 Understanding Modules for Managing Storage++++++++++++++++++++++++++++++++++
parted
lvg
lvol
filesystem
mount
vdo
++++++++++++++++++++++++++++++++++15.2 Implementing a Playbook to Manage Storage++++++++++++++++++++++++++++++++++

vim var/storage.yml
partition:
    -number:1
    start:1MiB
    end:1GiB
vgs:
    -name:vgweb
    devices:/dev/sdb

lvs:
    -name:lvwebdocs
    size:512M
    vgname:vgweb
    mount_point:/var/www

    -name:lvlogs
    size:256M
    vgname:vgweb
    mount_point:/var/log/httpd

vim storage-setup.yml
---
-name: storage setup
hosts:
vars_files:
    - var/storage.yml
tasks:
-name:verify partiton existance
parted:
device: /dev/sdb
state: present
number:"{{item.number}}"
part_start:"{{item.start}}"
part_end:"{{item.end}}"
loop:"{{partition}}"
```

```
-name:verify vg existance
lvg:
vg:"{{item.name}}"
pv:"{{item.devices}}"
loop:"{{vgs}}"

-name:create lv if not exist
lvol:
vg:"{{item.vgroup}}"
lv:"{{item.name}}"
size:"{{item.size}}"
loop:"{{lvs}}"
when: item.name not in ansible_lvm['lvs']

-name: verify xfs fs on all lv
filesystem:
dev: "/dev/{{item.vgroup}}/"{{item.name}}"
fstype:xfs
loop:"{{lvs}}"

-name:mount lv
mount:
path:"{{item.path}}"
src:"/dev/{{item.vgroup}}/"{{item.name}}"
fstype:xfs
state:mounted
loop:"{{lvs}}"


=======
identify new disk
create new vg 200mb research

========

++++++++++++++++++++++++++++++++++Lesson 15 Lab: Managing Storage++++++++++++++++++++++++++++++++++
detect all storage devices on the managed systems

---
-name: gather host storage devices
gather_facts:false
hosts: test01
vars:
disks: []
tasks:
-name:set fact
setup:
register: factone

-name:return data for parent block devices
set_fact:
disks:"{{disks + [item.key]}}"
loop:"{{all_devices}}"

-name:diag
debug:
msg:"{{disks}}"



##########################################################################################################################################
LESSON #16 -  MANAGEING NETWORKING
##########################################################################################################################################
++++++++++++++++++++++++++++++++++16.1 Using Network Roles for Network Management++++++++++++++++++++++++++++++++++
ansible-galaxy list
```

system roles are listed from /usr/share/ansible/roles

2 variables are used :
network_provider:nm
network_connections

system role should be used
/usr/share/ansible/roles
example in
/usr/share/doc/rhel-system-roles/network

vim setupnw.yml

-name: configure network
hosts: test
tasks:
- name: include vars
include_vars:
file: nwvars.yml
roles:
 - rhel-system-roles.network

vim nwvars.yml
network_provider:nm
network_connections:
-name:ens224
type:ethernet
ip:
address:192.168.4.22/24
zone:external
++++++++++++++++++++++++++++++++16.2 Understanding Modules for Network Management++++++++++++++++++++++++++++++++
nmcli:
hostname:
firewalld:

ansible all -m setup -a 'gather_subset=network filter=ansible_interfaces'
ansible all -m setup -a 'gather_subset=network filter=ansible_ens224'

vim setupnic.yml

-name:setup nic
hosts: test01
tasks:

-name:configure ens224
nmcli:
conn_name:new-conn
ifname:ens224
type:ethernet
ip4:10.0.0.10/24
state:present

-name:set hostname
hostname:
name:test01.example.com

-name:move ens224 to internal zone
firewalld:
zone:internal
permanenet:yes
immidiate:yes
interface:ens224
state:enabled

-name:enable http in firewall
firewalld:
zone: internal
service: http

```
permanent: yes
state:enabled

++++++++++++++++++++++++++++++++16.3 Using Ansible to Manage IPv6++++++++++++++++++++++++++++++++
vim ipv6.yml
---
-name:setup ipv6
hosts: test01
tasks:

-name:configure ens224
nmcli:
conn_name:ipv6-conn
ifname:ens224
type:ethernet
ip6:fc00::202/64
gw6:fc00::1
dns6:2001:4860:4860::8888
state:present


++++++++++++++++++++++++++++++++Lesson 16 Lab: Managing Networking++++++++++++++++++++++++++++++++
write a playbook to cehck if 2 interfaces are aviaable
if 2 are available then setup the second with te below ip
192.168.4.30/24

ansible test01 -m setup -a 'gather_subset=network filter=ansible_interfaces'

vim setupnw.yml

---
-name: configure network
hosts: test
tasks:
- name: include vars
roles:
 - rhel-system-roles.network


vim host_vars/test01/nwvars.yml
network_connections:
-name:ens224
type:ethernet
ip:
address:192.168.4.22/24


##########################################################################################################################################
LESSON #17 -  SAMPLE EXAM
##########################################################################################################################################
++++++++++++++++++++++++++++++++17.1 Exam Information++++++++++++++++++++++++++++++++
4 hours

++++++++++++++++++++++++++++++++17.2 Configuring the Basic Setup++++++++++++++++++++++++++++++++
5 centos8 servers

192.168.56.101  ansible1.example.com
192.168.56.102  ansible2.example.com
192.168.56.103  ansible3.example.com
192.168.56.104  ansible4.example.com
192.168.56.105  control.example.com

control.example.com 1gb ram, 20gb sda, ansible user with sshkeygen
```

```
ansible1.example.com    1gb ram, 20gb sda,5gb sdb
ansible2.example.com    1gb ram, 20gb sda,
ansible3.example.com    1gb ram, 20gb sda,5gb sdb
ansible4.example.com    1gb ram, 20gb sda,

root password ->password
perform all tasks in contol.example.com /home/ansible


#start
vagrant up
vagrant ssh

#stop
vagrant halt

#distroy
vagrant destroy

#after editing Vagrantfile
vagrant reload




ssh-copy-id -i ~/.ssh/id_rsa.pub remote-host

++++++++++++++++++++++++++++++++17.3 RHCE Sample Exam Assignments++++++++++++++++++++++++++++++++
------------task1-----------------------
configure control host with static inventory as well as ansible.cfg
ensure hosts can be reached via fqdn and hostname
configure the following in inventory

[test]
ansible1.example.com

[dev]
ansible2.example.com

[prod]
ansible3.example.com
ansible4.example.com

[servers:children]
dev
prod

SOLUTION

su - ansible
sudo yum install ansible -y
sudo yum install vim -y

vi $HOME/.vimrc
autocmd FileType yml setlocal ai ts=2 sw=2 et
set number




vi inventory

ansible1.example.com
ansible2.example.com
ansible3.example.com
ansible4.example.com

```

```
[test]
ansible1.example.com

[dev]
ansible2.example.com

[prod]
ansible3.example.com
ansible4.example.com

[servers:children]
dev
prod

ansible all -u root --ask-pass -m user -a 'name=ansible'
ansible all -u root --ask-pass -m authorized_key -a "user=ansible state=present key=\"{{lookup(\'file\',\'/home/ansible/.ssh/id_rsa.pub\')}}\""


vim ansible.cfg

[defaults]
inventory = ./inventory
remote_user = ansible
ask_pass = false

[privilege_escalation]
become = true
become_method = sudo
become_user = root
become_ask_pass = false




------------task2-----------------------
create setupreposerver.yml, setup contol as repo host,make sure meet the follwing
mount rhel8.iso on /var/ftp/repo
disable firewalld on control
enable and start vsftpd with anon access to /var/ftp/repo on control

------------task3-----------------------
create script to configure repos on manage hosts and perform following with adhoc command
disable exisiting repo
enable BaseOS and AppStream repo on control.example.com

------------task4-----------------------
configure manage hosts using setuphosts.sh. must complete the following
install python
useradd ansible and add in sudo test connectivity with adhoc command

------------task5 files-----------------------
use automated solution to create  the contents of /etc/hosts file  on all the managed hosts
based in information what was found from the inventory

------------task6 roles-----------------------
create custom role to cofigure ssh
-template for sshd_config, where SSH port, AllowedUsers and PermitRoot are managed through variables
-create a playbook which calls this role, before calling it should ensure selinux should be enforcing
- define variables to set SSH port to 2022 and AllowdUsers to user Ansible
- role should be called if os family is RedHat
- ensure firewall is open and appropriate SElinux context is set on the port

------------task playbook-----------------------
create playbook to setup ansible1 as web server, another plabook to install wget and elinks on ansile2 and 3
```

-create groups webserver and webclients and use them in the playbook
-provide a sample vhost.conf file as a jinja2 , copy it in web/templates dir and ensure its copied to all web servers
-define following variables and use them
web_package   httpd
web_service   httpd
web_config_file   /etc/httpd/conf/httpd.conf
firewall_service   http

-ensure web server package is installed, enalbed and started, port open in firewall
-copy httpd.conf template using following properties
    owher: root, group: root, permission mode:0644, SElinux context : httpd_config_t
- use handler to restart the webserver after coping the config file

create playobok that set up web client servers and install curl,wget on them
create a generic playbook web.yml which you can use to run both playbooks by using includes or imports

------------task roles-----------------------
vim galaxyroles.yml
use requirements file to install current versions geerlingguy.redis and geerlingguy.nginx roles
after runing this playbook above both roles should be present in ansible4.example.com

------------task file-----------------------
a playbook that generates motd file using a template, should print welcome message with below items
system memory, process count, primary disk size, secondary disk size (if not available then print NOT AVAILABLE)
only use ansible facts

------------task encrypt passwd-----------------------
vim userpw.yml
create user lisa with AES256 encoded passwd, make sure this password is included from and ansible vault encrypted file

------------task storage-----------------------
createvg.yml should create follwing
1GiB partitoin /dev/sdb1 on ansible1
2GiB partitoin /dev/sdb1 on ansible3
setup vg vgdata as well on both

setuplv.yml run on all hosts
lvdata of 2 gb- vgdata
if size insufficent then show error "Insuffient size" , if no vg then show error "vgdata does not exist"
------------task-----------------------
------------task-----------------------

++++++++++++++++++++++++++++++++++17.4 Setting up Ansible Solution++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++17.5 Managing File Content Solution++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++17.6 Working with Roles Solution++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++17.7 Creating Playbooks Solution++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++17.8 Installing Ansible Galaxy Roles Solution++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++17.9 Generating a File Solution++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++17.10 Creating a User with an Encrypted Password Solution++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++17.11 Managing Storage Solution++++++++++++++++++++++++++++++++++